Bank System Documentation

Overview

This program is a simple bank management system that allows users to create and manage their accounts. Users can perform the following operations:

- Create a new account.
- Deposit and withdraw money.
- Check account balance.
- View transaction history.
- Log in and out of their accounts.

Classes and Methods

1. Account Class

This class represents a bank account with attributes and methods to manage account operations.

Attributes:

- int accountNumber: Unique number for the account.
- String customerName: Name of the account holder.
- double balance: Current balance in the account.
- String accountType: Type of account (e.g., Savings or Current).
- List<String> transactionHistory: List of transactions performed on the account.

Constructors:

- Account(int accountNumber, String customerName, double balance, String accountType) Initializes the account with the given details and adds an entry to the transaction history.

Methods:

1. **void deposit(double amount)**
    - Adds the specified amount to the account balance.
    - Logs the transaction in the transactionHistory.
2. **void withdraw(double amount)**
    - Deducts the specified amount from the account balance if sufficient funds are available.
    - Logs the transaction in the transactionHistory.

    - Displays an error message if the balance is insufficient.
3. **void checkBalance()**
    - Prints the current account balance.
4. **void showTransactionHistory()**
    - Displays all transactions performed on the account.
    - Displays a message if there are no transactions.
5. **int getAccountNumber()**
    - Returns the account number.
6. **String getCustomerName()**
    - Returns the account holder's name.

Private Methods:

- **void addTransaction(String transaction)**
    - Adds a transaction description to the transactionHistory.

---

2. BankSystem Class

This class provides the user interface and logic for managing multiple accounts.

Attributes:

- static Map<Integer, Account> accounts: Stores all accounts using account numbers as keys.
- static Scanner scanner: Reads user input.
- static int currentAccountNumber: Tracks the currently logged-in account (-1 if no user is logged in).

Methods:

1. **public static void main(String[] args)**
    - The entry point of the program.
    - Continuously prompts the user to log in or perform operations until they choose to exit.
2. **private static void authenticateUser()**
    - Prompts the user to log in using their account number.
    - If the account exists, the user is authenticated, and the menu is displayed.

- If the account does not exist, the user can choose to create a new account.

3. **private static void createNewAccount()**
    - Collects the user's details to create a new account (name, initial balance, account type).
    - Generates a random account number.
    - Adds the account to the accounts map and logs the user in.
4. **private static void showAccountMenu()**
    - Displays a menu for the logged-in user with options to deposit, withdraw, check balance, view transaction history, logout, or exit.
    - Calls the respective methods from the Account class based on the user's choice.

Algorithm and Psuedo Code:

1. deposit(double amount)

Algorithm:

1. **Add the amount to the current balance.**
2. **Log the transaction with a description.**
3. **Display a success message.**

Pseudo-code:

```
Method deposit(amount):
  balance ← balance + amount
  addTransaction("Deposited: " + amount)
  Print "Deposited: " + amount
```

---

2. withdraw(double amount)

Algorithm:

1. **Check if amount is less than or equal to balance.**
    - **If true, subtract amount from balance.**
    - **Log the transaction with a description.**
    - **Display a success message.**

2. Otherwise, display an "Insufficient balance" message.

Pseudo-code:

```
Method withdraw(amount):
  If amount ≤ balance Then
    balance ← balance - amount
    addTransaction("Withdrawn: " + amount)
    Print "Withdrawn: " + amount
  Else
    Print "Insufficient balance."
```

---

3. checkBalance()

Algorithm:

1. Display the current balance.

Pseudo-code:

```
Method checkBalance():
  Print "Balance: " + balance
```

---

4. showTransactionHistory()

Algorithm:

1. If transactionHistory is empty, display "No transactions yet."
2. Otherwise, iterate through transactionHistory and print each transaction.

Pseudo-code:

```
Method showTransactionHistory():
  If transactionHistory is empty Then
    Print "No transactions yet."
  Else
    For each transaction in transactionHistory Do
```

Print transaction

---

5. addTransaction(String transaction)

Algorithm:

1. Append transaction to transactionHistory.

Pseudo-code:

```
Method addTransaction(transaction):
  transactionHistory.add(transaction)
```

---

6. createNewAccount()

Algorithm:

1. Prompt the user to input name, initial balance, and account type.
2. Generate a random account number.
3. Create a new Account object with the provided details.
4. Add the new account to the accounts map.
5. Log the user into the newly created account and display a success message.

Pseudo-code:

```
Method createNewAccount():
  name ← Input "Enter customer name:"
  balance ← Input "Enter initial balance:"
  accountType ← Input "Enter account type (Savings/Current):"
  accountNumber ← Random number
  newAccount ← Account(accountNumber, name, balance, accountType)
  accounts.put(accountNumber, newAccount)
  currentAccountNumber ← accountNumber
  Print "Account created successfully. Your account number is: " + accountNumber
```

---

7. authenticateUser()

Algorithm:

1. Prompt the user to input their account number.
2. If the account exists in the accounts map:
    - Set currentAccountNumber to the entered account number.
    - Display a welcome message with the customer's name.
3. Otherwise, prompt the user to create a new account or retry.

Pseudo-code:

```
Method authenticateUser():
  accountNumber ← Input "Enter Account Number:"
  account ← accounts.get(accountNumber)
  If account is not null Then
    currentAccountNumber ← accountNumber
    Print "Welcome, " + account.getCustomerName()
  Else
    Print "Invalid Account Number."
    choice ← Input "Do you want to create a new account? (yes/no):"
    If choice equals "yes" Then
      createNewAccount()
```

---

8. showAccountMenu()

Algorithm:

1. Display menu options.
2. Prompt the user to choose an option and execute the corresponding method:
    - 1: Deposit.
    - 2: Withdraw.
    - 3: Check Balance.
    - 4: View Transaction History.
    - 5: Logout.
    - 6: Exit the program.
3. Repeat until the user logs out or exits.

Pseudo-code:

```
Method showAccountMenu():
  Repeat
    Print "1. Deposit\n2. Withdraw\n3. Check Balance\n4. View Transaction History\n5. Logout\n6. Exit"
    choice ← Input "Enter choice:"
    Switch(choice):
      Case 1:
        amount ← Input "Enter amount to deposit:"
        account.deposit(amount)
      Case 2:
        amount ← Input "Enter amount to withdraw:"
        account.withdraw(amount)
      Case 3:
        account.checkBalance()
      Case 4:
        account.showTransactionHistory()
      Case 5:
        currentAccountNumber ← -1
        Print "Logged out successfully."
      Case 6:
        Print "Exiting..."
        Exit Program
      Default:
        Print "Invalid choice."
  Until choice equals 5
```

---

9. main()

Algorithm:

1. **Run a loop that continuously calls authenticateUser or showAccountMenu depending on the user's state (logged in or not).**

Pseudo-code:

```
Method main():
  Repeat
    If currentAccountNumber = -1 Then
      authenticateUser()
    Else
      showAccountMenu()
  Until program is terminated
```